

MASTER SQL COMMANDS MANUAL

Basic to Advanced Comprehensive Reference Guide (No Overlaps)

Aapki request ke mutabik dono references ko merge karke ek single structured list taiyar ki gayi hai. Is manual me se duplicate entries aur data overlaps ko poori tarah hta diya gaya hai taaki aapko ****Basic se Lekar Production-level Advanced Architecture**** tak ki ek clean sequential list mil sake.

1. Basic Commands (Database, Table & Constraints Management)

Yeh commands database framework create karne, schema columns define karne, aur initial parameters maintain karne ke liye hain:

A. DDL Core Blueprint Operations

```
-- New database definition structure
CREATE DATABASE IF NOT EXISTS db_name;

-- Standard table structures configuration with specific constraints
CREATE TABLE IF NOT EXISTS users (
  id INT NOT NULL PRIMARY KEY AUTO_INCREMENT, -- For MySQL style auto-gen
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  age INT CHECK(age > 6 AND age < 25),
  gender VARCHAR(255) DEFAULT 'other',
  journey_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- For PostgreSQL specific auto-increment setups
-- id SERIAL PRIMARY KEY

-- Removing database objects securely
DROP DATABASE IF EXISTS db_name;
DROP TABLE IF EXISTS users;
TRUNCATE TABLE users; -- Flushes all rows, preserves column blueprints
```

B. CLI / Shell Utility Reference

Dono environments ke utility shortcuts ko ek non-overlapping mapping list me format kiya gaya hai:

ADMINISTRATIVE REQUIREMENT	POSTGRESQL UTILITY	MYSQL SHELL STATEMENT
Available systems list checking	\1	SHOW DATABASES;

ADMINISTRATIVE REQUIREMENT	POSTGRESQL UTILITY	MYSQL SHELL STATEMENT
Select target database working terminal	<code>\c db_name</code>	<code>USE db_name;</code>
Active relation objects layout listing	<code>\dt</code>	<code>SHOW TABLES;</code>
Deep inspect table structural column data schemas	<code>\d table_name</code>	<code>DESCRIBE table_name;</code>
Close client shell interactive workspace	<code>\q</code>	<code>EXIT;</code>

2. Intermediate Commands (Core Manipulations & Functional Queries)

Yahan se aap exact row level manipulations, programmatic math queries, aur structured evaluation filtering operations perform karte hain:

A. Structural Row Manipulations (DML Core)

```
-- Targeting explicit entries insertion
INSERT INTO students (sid, sname, email) VALUES (1, 'Sahil', 'sahil@gmail.com');

-- Condition based modifier executions
UPDATE passenger SET name = 'Sahil' WHERE email LIKE '%gmail%';

-- Purging unwanted database record rows securely
DELETE FROM passenger WHERE id > 2 AND email LIKE '%yahoo%';
```

B. Mathematical, Aggregation & Character Mapping Strings

- **System Processing Math Logic:** Absolute variance tracker `ABS(col1 - col2)`, floating numbers modifications `ROUND(col, decimals)`, sequential boundaries scaling `CEIL(col)`, aur `FLOOR(col)`.
- **Character Formatting Functions:** Text mapping standardizer `UPPER(string)`, `LOWER(string)`, character merges strings `CONCAT(col1, col2)`, validation constraints tracker `LENGTH(string)`, aur text range cropper `SUBSTR(string, start, length)`.
- **Computational Data Aggregations:** `COUNT(DISTINCT col)`, `SUM(col)`, `MIN(col)`, `MAX(col)`, aur evaluation baseline tracker `AVG(col)`.

C. Order Execution Architecture Matrix

Database queries background compilation execution parameters standard constraints logical layout tracking rules:

```
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
```

```
-- Comprehensive sequential application example
SELECT star, AVG(votes) AS opening
```

```
FROM movies
WHERE genre IN ('Action', 'Horror', 'Drama')
GROUP BY star
HAVING AVG(votes) > 1000
ORDER BY opening DESC
LIMIT 10;
```

3. Advanced Commands (Table Modifiers, Logic Streams & Optimization)

Enterprise scalable operations, logic evaluation processing blocks, queries pipeline debugging, aur structural engineering changes:

A. Dynamic Table Modifications (ALTER Commands Setup)

SCHEMA OBJECTIVE	POSTGRESQL SYNTAX RULE	MYSQL SYNTAX RULE
Inject new attribute columns	ALTER TABLE students ADD COLUMN college VARCHAR(255) NOT NULL;	
Purge existing structural columns	ALTER TABLE students DROP COLUMN age;	
Alter schema target datatype definitions	ALTER TABLE students ALTER COLUMN sname TYPE INT;	ALTER TABLE students MODIFY COLUMN sname INT;

B. Conditional Logic Flows & Multi-Table Joins

```
-- Logical evaluation evaluations processing streams
SELECT title, (gross - budget) AS profit,
CASE
    WHEN (gross - budget) > 1000000000 THEN 'SUPER HIT'
    WHEN (gross - budget) > 0 THEN 'AVERAGE'
    ELSE 'FLOP'
END AS verdict FROM movies;

-- Cross relational table unions sets matching
SELECT id FROM users UNION SELECT gid FROM groups; -- Evaluates distinct entries across domains
```

C. Pipeline Debugging & Native Connections Set

- **Execution Optimization Profiler:** Query background path blueprints runtime tracking metrics analyser engine:

EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'sahil@example.com';
- **Relational Full Merge Set Mapping (PostgreSQL Native vs MySQL Simulation Workaround):**

```
-- PostgreSQL Engine Native Run
SELECT * FROM membership m FULL OUTER JOIN users u ON m.uid = u.id;

-- MySQL Workaround Simulation Model Setup
SELECT * FROM membership m LEFT OUTER JOIN users u ON m.uid = u.id
UNION
SELECT * FROM membership m RIGHT OUTER JOIN users u ON m.uid = u.id;
```

4. Production-Grade Enterprise Concepts (Architecture Layer)

Industrial applications high-availability structures, safety configurations, transaction states, isolation protocols and auto data recovery setups:

A. Safe Transactions Blocks Processing

```
-- Isolated transaction state parameters setup
BEGIN; -- Starts operation runtime framework block (MySQL uses: START TRANSACTION;)
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT; -- Changes permanently flushed inside structural disk files
-- If crash checkpoints match or error logs pop up: ROLLBACK;
```

B. Database Views & Materialized Cache Blocks Virtualization

- **Standard Views Definition:** Virtual mapping engine queries shortcut mask wrapper.

```
CREATE VIEW active_users AS SELECT name, email FROM users WHERE status = 'active';
```

- **PostgreSQL Materialized View Caching Model:** Computational resource load management query outputs snapshots disk indexing.

```
CREATE MATERIALIZED VIEW mv_sales_summary AS
SELECT product_id, SUM(amount) FROM sales GROUP BY product_id;

-- Force system evaluation pipeline calculations updates cache:
REFRESH MATERIALIZED VIEW mv_sales_summary;
```

C. Storage Defragmentation Mechanics & Automated Maintenance Purge

Dead entries records updates metadata index allocations physical alignments refresh optimization systems execution configurations:

```
-- PostgreSQL Engine Tuple Space Purge Framework
VACUUM FULL table_name; -- Force releases unallocated spaces chunks directly back to OS layout
ANALYZE table_name; -- Recalculates metrics parameter planning coefficients inside query optimization charts

-- MySQL Tables Defragmentation Optimization Code
OPTIMIZE TABLE table_name; -- Re-aligns internal index structure branches to eliminate fragmentation lags
```

D. Upsert Logic Execution (Conflict Overrides)

```
-- PostgreSQL Target Identity Conflict Override Setup
INSERT INTO users (id, name) VALUES (1, 'Sahil')
ON CONFLICT (id) DO UPDATE SET name = EXCLUDED.name;

-- MySQL Target Duplication Overwrite Rules Framework
INSERT INTO users (id, name) VALUES (1, 'Sahil')
ON DUPLICATE KEY UPDATE name = VALUES(name);
```